



Advanced Multi-Tenant E-Commerce SaaS Platform

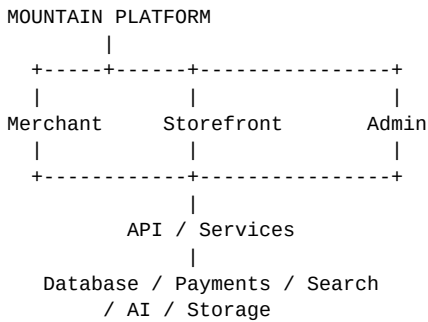
Product & Technical Specification — Client, Merchant, Admin, AI, Infrastructure & Scalability

Vision: Build Mountain as a serious commerce operating system where merchants can create, customize and operate complete online stores from one platform.

Document scope: Product architecture, applications, modules, data model, APIs, security, payments, AI, analytics, infrastructure, DevOps and phased development strategy.

1. Product Vision

Mountain is a multi-tenant SaaS platform inspired by the store-building model of Shopify. One platform serves many independent merchants, while each merchant receives an isolated store, catalog, customers, orders, inventory, settings and analytics.



2. Applications

2.1 Mountain Merchant

- Account and profile management
- Store creation and configuration
- Products, variants and categories
- Inventory and warehouses
- Orders and customers
- Discounts and marketing
- Shipping and payments
- Analytics
- AI tools
- Employees and permissions
- Subscription and billing

2.2 Mountain Storefront

- Public store generated for each merchant
- Home, catalog, categories and product pages
- Search and filters
- Cart and checkout
- Customer accounts
- Wishlist and reviews
- Order tracking
- Responsive/mobile-first design
- Custom domains

2.3 Mountain Admin

- Global platform dashboard

- Merchant and user management
- Subscriptions, invoices and transactions
- Support and tickets
- Security and audit logs
- Fraud monitoring
- System health and API monitoring
- AI usage and cost monitoring
- Marketing and platform settings

3. Multi-Tenant Architecture

Mountain must be designed as a multi-tenant SaaS. Each merchant is represented by a tenant, and every tenant-owned resource must be isolated using tenant-aware authorization and database queries.

```
Tenant
|
+-- Users
+-- Store
+-- Products
+-- Customers
+-- Orders
+-- Inventory
+-- Employees
+-- Settings
```

```
Every tenant-owned record:
{
  id,
  tenantId,
  ...
}
```

Critical rule: never query tenant-owned resources without resolving and validating the current tenant ID.

4. Roles & Permissions

Platform roles

- SUPER_ADMIN
- ADMIN
- FINANCE_ADMIN
- SUPPORT_ADMIN
- SECURITY_ADMIN
- MARKETING_ADMIN
- ANALYTICS_ADMIN

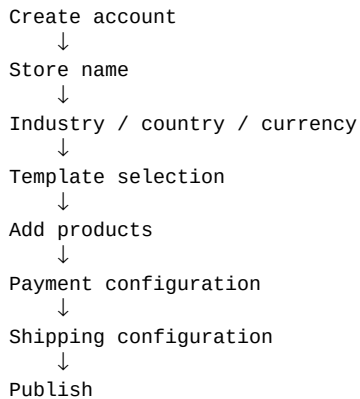
Merchant roles

- OWNER
- MANAGER
- STAFF
- ACCOUNTANT

- MARKETING
- FULFILLMENT
- CUSTOMER_SUPPORT

Use RBAC with granular permissions such as `products.read`, `products.create`, `orders.read`, `orders.update`, `orders.refund`, `analytics.read` and `settings.update`.

5. Store Creation



- Automatic store setup
- Theme and page initialization
- Navigation generation
- Policy pages
- Checkout configuration
- SEO defaults
- Domain setup

6. Store Builder

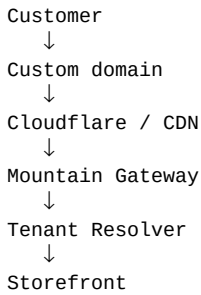
A component-based drag-and-drop builder allows merchants to create pages without coding.

- Hero
- Banner
- Product grid
- Product carousel
- Categories
- Testimonials
- Reviews
- FAQ
- Newsletter
- Video
- Image
- Text

- Countdown
- Social media
- Featured products
- Best sellers
- Custom HTML

Pages should be represented as structured components/configuration rather than hardcoded templates.

7. Custom Domains



- DNS verification
- SSL/HTTPS
- Domain routing
- Custom domains and subdomains
- Tenant resolution

8. Product System

- Product title and description
- SKU
- Brand
- Category
- Tags
- Images and videos
- Variants
- Pricing
- Inventory
- Shipping information
- SEO metadata
- Reviews
- Custom metadata

Variants

Products can have dimensions such as size, color and material. Each combination can have its own SKU, price and stock.

9. Inventory & Warehouses

- Stock
- Reserved
- Available
- Incoming
- Damaged
- Returned
- Multiple warehouses
- Low-stock alerts
- Inventory synchronization

A typical availability formula is: $\text{Available} = \text{Stock} - \text{Reserved}$.

10. Orders

PENDING → CONFIRMED → PROCESSING → SHIPPED → DELIVERED
■ CANCELLED / REFUNDED / RETURNED / FAILED

- Customer
- Products and variants
- Quantity
- Subtotal
- Discount
- Tax
- Shipping
- Total
- Payment
- Addresses
- Tracking
- Order timeline

11. Payments

Create a provider abstraction so Mountain is not tightly coupled to one payment provider.

- Stripe
- PayPal
- Bank transfer
- Cash on delivery
- Local payment providers

Payment provider interface: `createPayment()`, `capturePayment()`, `refundPayment()`, `cancelPayment()`, `getPaymentStatus()`.

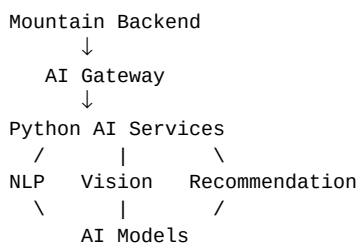
12. Mountain Business Model

- Free
- Starter
- Pro
- Business
- Enterprise

Potential revenue sources include subscriptions, transaction fees, premium features, AI usage and extra storage. Pricing should be configurable rather than hardcoded into the core application.

13. AI Platform

- AI product title and description generation
- SEO content generation
- Product image enhancement
- Background removal
- AI store generation
- Sales analysis
- Recommendations
- Demand forecasting
- Fraud detection
- Customer segmentation
- AI chatbot
- Marketing assistance



14. AI Store Builder

A merchant can describe the desired business in natural language, for example: "Create an online clothing store for young men." The system can prepare a theme, colors, homepage structure, categories, navigation, product sections, marketing copy and SEO defaults. The merchant remains in control and approves the generated result.

15. AI Analytics Assistant

The assistant should answer business questions using actual store data such as sales, orders, products, traffic, customers and inventory. It should distinguish measured data from generated explanations and avoid inventing metrics.

16. Search

- Full-text search
- Autocomplete
- Filters
- Facets
- Typo tolerance
- Synonyms
- Ranking
- Recommendations
- Trending products

Recommended architecture: MongoDB as source of commerce data, with OpenSearch/Elasticsearch as a dedicated search index.

17. Recommendation Engine

Recommendations can use views, clicks, searches, purchases, carts, wishlists and categories.

- You may also like
- Frequently bought together
- Recommended for you
- Similar products
- Trending products

18. Analytics

- Revenue
- Orders
- Customers
- Conversion rate
- Average order value
- Traffic
- Sessions
- Cart abandonment
- Returning customers
- Product performance

Visitors
↓
Product Views
↓
Add to Cart
↓
Checkout
↓
Purchase

19. Cart & Customer Accounts

- Guest cart
- Authenticated cart
- Persistent cart
- Cart merging after login
- Coupons
- Variants
- Quantity validation
- Inventory validation
- Shipping estimation
- Profile
- Addresses
- Orders
- Wishlist
- Reviews
- Returns
- Saved payment methods
- Notifications

20. Reviews

- Rating
- Text
- Images
- Verified purchase
- Helpful votes
- Merchant response
- Moderation
- Reporting

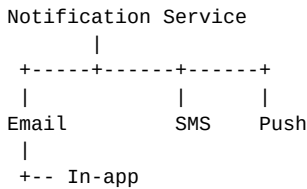
Moderation states: PENDING, APPROVED, REJECTED, REPORTED.

21. Marketing

- Coupons
- Discount codes
- Automatic discounts
- Flash sales
- Bundles

- Free shipping
- Buy X Get Y
- Email campaigns
- Push notifications
- Abandoned-cart campaigns

22. Notifications



- Order created
- Payment successful
- Order shipped
- Order delivered
- Refund
- Low inventory
- New customer
- New review
- Subscription events

23. Security

- OAuth2/JWT authentication
- Refresh tokens
- 2FA
- RBAC
- Rate limiting
- API gateway
- Input validation
- Encryption
- Secrets management
- Audit logs
- Session management
- CSRF protection
- CORS policy
- Security headers
- Password hashing with Argon2id

Never store raw payment card details. Use compliant payment providers and tokenized payment flows.

24. Fraud Detection

- IP/device signals
- Transaction velocity
- Failed payments
- Country/location signals
- Transaction amount
- Multiple-account patterns
- Suspicious order behavior

Use the fraud engine to route transactions to normal processing, additional review or blocking according to configurable rules.

25. Backend Architecture

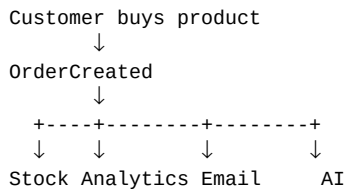
Start with a modular monolith rather than immediately creating many microservices. Keep domains separated so high-load components can later be extracted.

```
Backend
████ Auth
████ Users
████ Tenants
████ Stores
████ Products
████ Categories
████ Inventory
████ Orders
████ Payments
████ Customers
████ Reviews
████ Discounts
████ Shipping
████ Notifications
████ Analytics
████ Search
████ AI
████ Billing
████ Admin
```

26. Event-Driven Architecture

Use domain events for asynchronous work and future service separation.

- OrderCreated
- PaymentCompleted
- ProductCreated
- ProductUpdated
- InventoryChanged
- CustomerCreated
- OrderShipped
- OrderDelivered



27. Caching

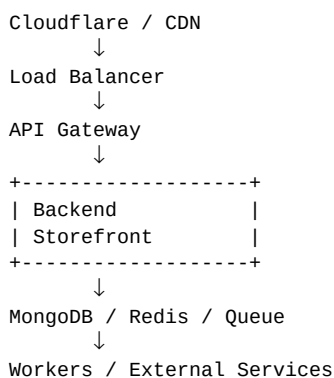
Redis can be used for sessions, carts, product/store configuration caching, rate limiting and temporary data.

28. Data Architecture

Initial stack can use MongoDB for flexible commerce data. At larger scale, specialized storage can be introduced.

- PostgreSQL — transactional/financial data where appropriate
- MongoDB — flexible commerce/catalog data
- Redis — cache and sessions
- OpenSearch — search
- S3-compatible storage — images/files
- ClickHouse — high-volume analytics

29. Cloud Architecture



- Docker
- Kubernetes when scale justifies it
- CI/CD
- Infrastructure as Code with Terraform
- Object storage
- CDN
- Secrets management

30. Mobile Applications

Mountain Merchant

- Dashboard
- Orders

- Products
- Customers
- Analytics
- Notifications

Mountain Customer

- Store discovery
- Search
- Cart
- Checkout
- Orders
- Wishlist

Flutter can provide one cross-platform codebase for Android and iOS.

31. Recommended Technology Stack

Layer	Technology
Merchant Web	React + TypeScript
Storefront	Next.js
Admin	React + TypeScript
Backend	Node.js + NestJS
Database	MongoDB
Cache	Redis
Search	OpenSearch
Queue/Eventing	Kafka or RabbitMQ
Storage	S3-compatible object storage
Payments	Stripe + local providers
AI	Python + FastAPI
AI Models	PyTorch / TensorFlow
Mobile	Flutter
Auth	OAuth2 + JWT + 2FA
API	REST + WebSocket
Containers	Docker
Infrastructure	Kubernetes
CI/CD	GitHub Actions
Observability	OpenTelemetry + metrics/logging stack
CDN/Security	Cloudflare

32. Project Structure

```
mountain/  
  ■■■ apps/  
  ■ ■■■ merchant/
```

```
■ ■■■ storefront/
■ ■■■ admin/
■ ■■■ mobile/
■■■ services/
■ ■■■ api/
■ ■■■ ai/
■ ■■■ worker/
■ ■■■ search/
■ ■■■ notifications/
■■■ packages/
■ ■■■ ui/
■ ■■■ types/
■ ■■■ config/
■ ■■■ auth/
■ ■■■ utils/
■■■ infrastructure/
■ ■■■ docker/
■ ■■■ kubernetes/
■ ■■■ terraform/
■■■ docs/
```

33. Core Data Entities

- User
- Tenant
- Store
- Domain
- Theme
- Page
- Product
- ProductVariant
- Category
- Inventory
- Warehouse
- Customer
- Cart
- CartItem
- Order
- OrderItem
- Payment
- Refund
- Shipment
- Coupon
- Discount
- Review
- Wishlist
- Subscription

- Invoice
- Notification
- AuditLog
- Employee
- Role
- Permission
- AIRequest
- AnalyticsEvent

34. Admin Command Center

- Platform overview
- Users, merchants and stores
- Customers
- Orders and products
- Subscriptions and payments
- Revenue and platform metrics
- Support tickets
- Security and audit logs
- Infrastructure health
- API performance
- AI usage and costs

Merchant detail should provide a controlled operational view of the merchant's store, subscription, activity and support history.

35. Support System

- Tickets
- Conversations
- Attachments
- Priority
- Status
- Assigned agent
- Internal notes

Suggested statuses: OPEN, IN_PROGRESS, WAITING_CUSTOMER, RESOLVED, CLOSED.

36. Internationalization

- French
- English

- Arabic
- RTL support
- Locale-aware dates and numbers
- Currencies
- Time zones
- Tax rules

The platform should be localization-ready from the first version rather than adding translation architecture later.

37. Multi-Currency & Tax

Store and customer contexts may determine currency, language, tax and shipping rules. Currency and tax logic should be isolated services/modules rather than hardcoded into checkout.

38. Quality & DevOps

- Unit tests
- Integration tests
- End-to-end tests
- API tests
- Security tests
- Load tests
- Pull-request checks
- Dependency/security scanning
- Docker builds
- Staging deployment
- Automated E2E
- Production deployment
- Observability

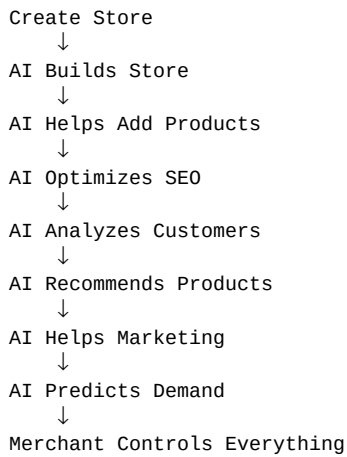
39. SaaS Metrics

- MRR
- ARR
- Active merchants
- Active stores
- New stores
- Churn
- Subscription upgrades/downgrades
- GMV
- Orders

- Average revenue per merchant
- AI usage
- Infrastructure cost

40. Long-Term Positioning

Mountain can evolve from a store builder into an AI-powered commerce operating system.



41. Development Roadmap

Phase 1 — Core

- Authentication
- Multi-tenancy
- Store creation
- Products
- Categories
- Cart
- Checkout
- Orders
- Merchant dashboard
- Admin dashboard

Phase 2 — Professional

- Themes
- Page builder
- Custom domains
- Payments
- Shipping
- Inventory
- Coupons
- Reviews

- Analytics
- Notifications

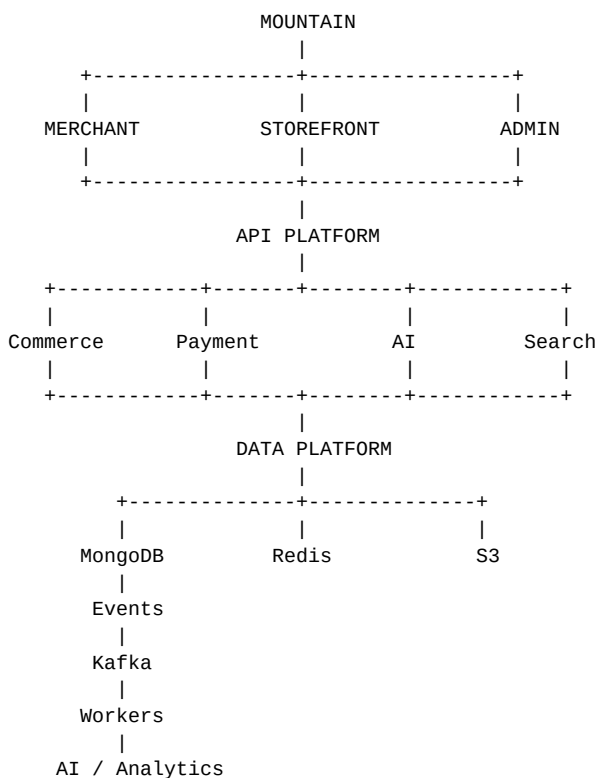
Phase 3 — Advanced

- Search
- Recommendations
- Multi-warehouse
- Subscriptions
- Mobile apps
- Event-driven architecture
- Advanced analytics

Phase 4 — AI

- AI Store Builder
- AI Product Generator
- AI SEO
- AI Analytics
- AI Recommendations
- AI Marketing
- AI Customer Assistant
- AI Forecasting

42. Final Architecture



43. Product Principle

Build Mountain as a platform, not a website. The core architecture should make it possible for many merchants to independently operate stores while Mountain centrally provides identity, commerce infrastructure, payments, analytics, AI, security and administration.

The architecture is intentionally designed so the first production release can start as a modular monolith and progressively extract high-load domains into services as usage grows.